Author Instructions for IMVIP 2019

Michael McDonagh

Anonymous Affiliation

Abstract

This study explores the development of a deep learning system to classify 19 different types of food. Recognising food from images is a difficult task because many dishes look very similar and the way a meal is plated can vary significantly. This research compares two main methods: a custom-built Convolutional Neural Network (CNN) and a Transfer Learning approach that uses a pre-trained model as its foundation.

Keywords: Imaging, Image Processing, Machine Vision, etc. (Maximum five)

1 Introduction

Initial attempts with a custom model resulted in clear underfitting, where the accuracy was no better than random guessing at 5.2%. By improving the model structure, specifically by using double convolutional blocks, increasing the number of filters to 128, adding Dropout Layers to prevent over-reliance on specific features, the custom model eventually achieved a stable validation accuracy of 65%. To try and improve further, a two-phase transfer learning strategy was used. This involved an initial phase where the base model was frozen, followed by a fine-tuning phase to adapt the weights to this specific food dataset.

Figure 1: Introduction Banner.

2 Data and Pre-Processing

The first thing I did was to initialise any of the variables I will be using that will detail how I will be processing my data. The seed that I set is `421944` for my student number, this will allow for reproducability of my results. I then have the batch sizes for my data in batches of `32`, dealing with image sizes of `128x128` and `64x64`. Computational tasks were performed locally using an NVIDIA GeForce RTX 4060 GPU, with a verification step included in the script to confirm CUDA acceleration was active.

2.1 Dataset Description

The image dataset being used is a subset of the Food 101 dataset containing 19 folders named after the type of food contained in its images. The images in use are `512x512`, but will be rescaled across `128x128` and `64x64`, grayscaled and/or augmented for the purpose of exploring different hyperparameters and comparing models.

2.1.1 Preprocessing and Splits

The dataset contained images with 3 colour channels (RGB), since that would involve numbers ranging from 0.0 to 255, it was important the training data was normalised. Normalisation will help with data moving between layers, and help prevent vanishing/exploding gradients. The training split that was chosen was to split the Training Set at 30% (13,312 images), Validation 10% (1,888 images), for tuning the hyperparameters and the final Test Set at 20% (3,840 images). The final Test Set is reserved to the final evaluation of the model.

- Training Set at 30% (13,312 images) - Used for model weight updates
- Validation 10% (1,888 images) - Hyperparameter Tuning
- Test Set at 20% (3,840 images) - Reserved to the final evaluation of the model.

2.1.2 Exploratory Analysis

An initial audit of the dataset confirmed it is perfectly balanced. Each of the 19 classes contains exactly $1,000$ images, totaling $19,000$ samples with 3 channels and 128x128 in size. This eliminates the risk of class imbalance, where a model might become biased toward a majority class to artificially inflate accuracy. In such a case where there was an imbalance, class weights can be introduced to give equal importance to all classes.

2.2 Experiments

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

2.3 CNN From Scratch: Iterative Development

The development of the custom Convolutional Neural Network (CNN) followed an iterative progression, transitioning from a high-parameter baseline to a refined, efficient architecture optimized for generalization.

2.3.1 Initial Models: From Overfitting to Underfitting

The development began with **Model V1 (Baseline)**, which utilized two convolutional blocks and a single MaxPooling2D layer for downsampling. This model exhibited extreme overfitting, achieving a training accuracy of 99% while validation accuracy stagnated at 20%. This massive disparity indicated that the network was memorizing noise within the training set rather than learning generalizable features.

To mitigate this, **Model V2** introduced three convolutional blocks with doubled layers per block to extract more granular features. A dense layer of 256 units and a Dropout rate of 0.5 were added to reduce feature dependency. However, this resulted in an over-correction into underfitting, with both training and validation accuracies dropping below 6%.

2.3.2 Structural Refinement and Regularization

Model V3 aimed to find a middle ground by reverting to two convolutional blocks while introducing **Batch Normalization** to stabilize internal covariate shift. A significant architectural change was the replacement of the flattening layer with **GlobalAveragePooling2D**, which reduced the parameter count from approximately 8.6 million to just 41,299. This led to smoother convergence and a more aligned relationship between training and validation metrics.

In **Model V4**, **Image Augmentation** (horizontal flips, 10% rotations, and 10% zooms) was integrated into the pipeline to artificially expand the dataset diversity. While this stabilized the training loss, the model remained sensitive to extreme transformations, resulting in a validation accuracy of approximately 33.2%.

2.3.3 Scaling and Final Optimization

Models **V5** and **V6** focused on scaling depth and width to capture complex semantic features. **Model V5** increased the maximum filter count to 256, while **Model V6** pushed the feature extractor to 1024 filters with a 2048-unit dense layer. This "brute force" scaling allowed **Model V6** to reach a validation accuracy of 66.9%.

Finally, **Model V7** was developed to prioritize efficiency over raw size. By reducing the dense layer from 2048 to 128 units and implementing a `ReduceLROnPlateau` scheduler, the model achieved a **44% reduction in validation loss** compared to V6. Despite a slightly lower peak training accuracy, V7 demonstrated superior generalization by maintaining a validation accuracy of 64.6% with significantly higher confidence (lower loss).

Ultimately, **Model V7** was selected as the superior architecture built from scratch. It provided the optimal balance of depth and regularization, serving as the final benchmark before transitioning to transfer learning techniques.

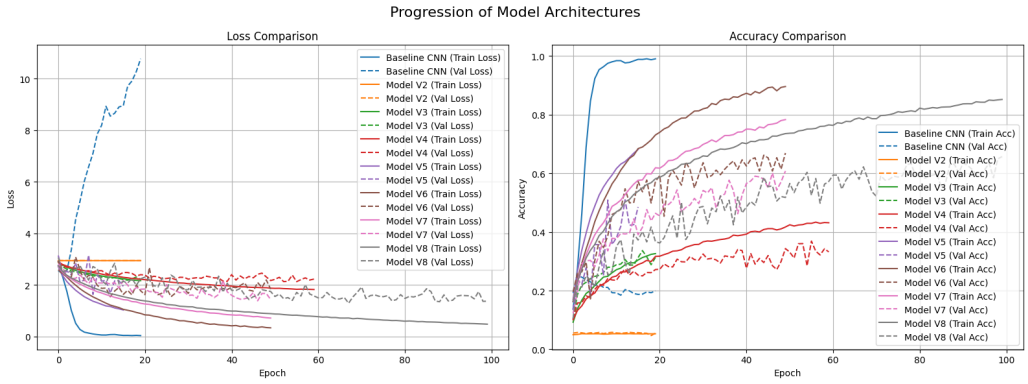

Figure 2: Performance Comparison of CNN Architectures from Scratch

3 Transfer Learning Analysis

Transfer learning was implemented to evaluate if a model pre-trained on a large-scale dataset (ImageNet) could outperform the custom CNN architectures previously developed. This approach allows the model to leverage complex feature extractors that have already learned generic visual patterns like edges, textures, and shapes.

3.1 Model Selection: MobileNetV2

MobileNetV2 [1] was chosen as the base architecture over larger models like VGG16 or ResNet50 for several key reasons:

Table 1: Detailed Model Performance Comparison. Improvement metric is based on comparison with Model V2 (128x128).

Model Version	Epochs	Total Time (s)	Time/Ep (s)	Train Acc	Val Acc	Val Loss	Improv. (%)
Model V2 (128x128)	20.0	307.29	15.36	0.0522	0.0546	2.9450	0.00
Model V2 (64x64)	20.0	114.45	5.72	0.7214	0.2834	3.5272	22.88
Model V3 (128x128)	40.0	308.08	7.70	0.4235	0.4105	1.9015	35.59
Model V3 (64x64)	40.0	128.58	3.21	0.4153	0.3914	1.9181	33.68
Model V4 (128x128)	60.0	811.30	13.52	0.4330	0.2823	2.3407	22.77
Model V4 (64x64)	60.0	305.11	5.09	0.4168	0.3242	2.1959	26.96
Model V5 (128x128)	27.0	446.51	16.54	0.7571	0.5117	2.0990	45.71
Model V5 (64x64)	27.0	174.69	6.47	0.6880	0.5657	1.5755	51.11
Model V6 (128x128)	50.0	1050.54	21.01	0.8991	0.6112	2.2366	55.66
Model V6 (64x64)	50.0	389.71	7.79	0.8441	0.5021	2.5986	44.75
Model V7 (128x128)	30.0	741.33	20.04	0.8135	0.6462	1.2479	59.16
Model V7 (64x64)	30.0	218.31	7.04	0.7343	0.5673	1.5037	51.27
V7 Gray (128x128)	30.0	517.91	19.18	0.6858	0.5132	1.5896	45.86
V7 Gray (64x64)	30.0	138.01	6.27	0.5730	0.4460	1.8938	39.14

- **Efficiency:** MobileNetV2 is designed for mobile and resource-constrained environments, making it highly efficient for the 128x128 input size used in this project.
- **Parameter Economy:** The base model contains **2,257,984 parameters**, yet it allows for a "frozen" state where **0 parameters** are trainable during the initial phase. This prevents the destruction of pre-trained weights while training the new classification head.
- **Performance vs. Size:** It provides a high accuracy-to-parameter ratio, which is critical for maintaining fast training times without sacrificing depth.

3.2 Phase 1: Feature Extraction

In the first phase, the MobileNetV2 base is used strictly as a feature extractor. The following architecture was implemented:

- **Preprocessing:** The `preprocess_input` function is critical as it scales input pixels to the range $[-1, 1]$, which is the specific requirement for MobileNetV2 to function correctly.
- **Frozen Base:** The base model is called with `training=False`, ensuring that Batch Normalization layers continue to use the statistics learned from ImageNet rather than the small batch statistics of the current dataset.
- **Global Average Pooling (GAP):** Instead of a Flatten layer, GAP was used to reduce the spatial dimensions of the feature maps, significantly reducing the parameter count in the dense layers and helping to prevent overfitting.
- **Custom Head:** A dense layer of 128 units with ReLU activation was added, flanked by two **Dropout layers (0.3)** to ensure robust regularization.

Using a standard learning rate of 1×10^{-3}, this phase achieved a **best validation accuracy of 72.35%**.

3.3 Phase 2: Fine-Tuning

To further optimize performance, a fine-tuning phase was initiated. The top layers of the base model were unfrozen to allow them to adapt to the specific nuances of the food dataset.

- **Selective Unfreezing:** Out of the **154 total layers** in MobileNetV2, only the **top 20 layers** were made trainable. This keeps the early "general" feature detectors intact while specializing the deeper "complex" detectors.
- **Learning Rate Strategy:** The learning rate was reduced to 1×10^{-5}. This 100x reduction is essential to avoid large weight updates that could degrade the pre-trained feature representations.

During this phase, the validation accuracy reached **71.45%**. While slightly lower than Phase 1, the training curves in image_6b96b3.png show that the validation loss remained stable around 0.9, indicating the model was well-regularized.

3.4 Comparison with Custom CNN (Model V7)

The transfer learning approach significantly outperformed the best custom CNN model (V7).

Table 2: Comparison Between Custom CNN and Transfer Learning

Model Type	**Validation Acc (%)**	**Validation Loss**	**Status**
Custom CNN (V7 128x128)	64.62	1.2479	Best Custom
MobileNetV2 (Phase 1)	**72.35**	**0.90**	**Overall Best**

The transition to transfer learning resulted in an approximate **7.73% absolute increase in validation accuracy** and a substantial reduction in loss compared to the manual architecture. This demonstrates that for this dataset, the pre-trained features of MobileNetV2 are superior to those learned from scratch by the custom CNN.

3.5 Results and Comparison

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

3.6 Real-World Test

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

3.7 Conclusion

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

3.8 Bibliographic references

References

[1] TensorFlow Authors. tf.keras.applications.mobilenetv2. `https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2`, 2024. Accessed: 2024-05-20.